

AUTHORIZED KEYWORD SEARCH OVER ENCRYPTED DATA IN CLOUD ENVIRONMENT

A Major Project Report

Submitted to



Jawaharlal Nehru Technological University, Hyderabad
*In partial fulfillment of the requirements for the
award of the degree of*

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING
By

DOOLAM NAVYA	- 20VE1A05D2
GUDI SAITEJA	- 20VE1A05D8
KARRA VAMSHIDHAR REDDY	- 20VE1A05E3
SAI NANDINI NAENI	- 20VE1A05G6

Under the Guidance
of
Mrs.T.V DIVYA
ASSISTANT PROFESSOR



SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Affiliated to JNTUH, Approved by A.I.C.T.E and Accredited by NAAC, New Delhi)
Bandlaguda, Beside InduAranya, Nagole,
Hyderabad-500068, Ranga Reddy Dist.
(2020-2024)



SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major Project Report on **“AUTHORIZED KEYWORD SEARCH OVER ENCRYPTED DATA IN CLOUD ENVIRONMENT”** submitted by **DOOLAM NAVYA, GUDI SAITEJA, KARRA VAMSHIDHAR REDDY, SAI NANDINI NAENI** bearing Hall ticket numbers: **20VE1A05D2, 20VE1A05D8, 20VE1A05E3, 20VE1A05G6** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** from Jawaharlal Nehru Technological University, Kukatpally, Hyderabad for the academic year 2023-2024 is a record of bonafide work carried out by them under our guidance and Supervision.

Project Coordinator

Dr.U.M.Fernandes Dimlo

HOD & Professor

Head of the Department

Dr.U.M.Fernandes Dimlo

Associate Professor

Internal Guide

Mrs.T.V Divya

Assistant Professor

External Examiner



SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We **DOOLAM NAVYA, GUDI SAITEJA, KARRA VAMSHIDHAR REDDY, SAI NANDINI NAENI** bearing Hallticket numbers: **20VE1A05D2, 20VE1A05D8, 20VE1A05E3, 20VE1A05G6** hereby declare that the Major Project titled **AUTHORIZED KEYWORD SEARCH OVER ENCRYPTED DATA IN CLOUD ENVIRONMENT** done by us under the guidance of **Mrs.T.V DIVYA , Assistant Professor** which is submitted in the partial fulfillment of the requirement for the award of the B.Tech degree in **Computer Science and Engineering** at **Sreyas Institute of Engineering and Technology** for **Jawaharlal Nehru Technological University, Hyderabad** is our original work.

Doolam Navya	-20VE1A05D2
Gudi Saiteja	- 20VE1A05D8
Karra Vamshidhar Reddy	- 20VE1A05E3
Sai Nandini Naeni	- 20VE1A05G6

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without mention of the people who made it possible through their guidance and encouragement crowns all the efforts with success.

We take this opportunity to acknowledge with thanks and deep sense of gratitude to **Mrs.T.V Divya, Assistant Professor, Department of Computer Science and Engineering** for her constant encouragement and valuable guidance during the Project work.

A Special vote of Thanks to **Dr.U.M.Fernandes Dimlo, Head of the Department and Project Coordinator** who has been a source of Continuous motivation and support. They had taken time and effort to guide and correct us all through the span of this work.

We owe very much to the **Department Faculty, Principal** and the **Management** who made our term at Sreyas Institute of Engineering and Technology a stepping stone for our career. We treasure every moment we had spent in college.

Last but not the least, our heartiest gratitude to our parents and friends for their continuous encouragement and blessings. Without their support this work would not have been possible.

Doolam Navya - 20VE1A05D2

Gudi Saiteja - 20VE1A05D8

Karra Vamshidhar Reddy - 20VE1A05E3

Sai Nandini Naeni - 20VE1A05G6

ABSTRACT

For better data availability and accessibility while ensuring data secrecy, end-users often tend to outsource their data to the cloud servers in an encrypted form. However, this brings a major challenge to perform the search for some keywords over encrypted content without disclosing any information to unintended entities. This paper proposes a novel expressive authorized keyword search scheme relying on the concept of ciphertext-policy attribute-based encryption. The originality of the proposed scheme is multifold. First, it supports the generic and convenient multi-owner and multi-user scenario, where the encrypted data are outsourced by several data owners and searchable by multiple users. Second, the formal security analysis proves that the proposed scheme is semantically secure against chosen keyword and outsiders keyword guessing attacks. Third, an interactive protocol is introduced which avoids the need of any secure-channels between users and service provider. Fourth, due to the concept of bilinear-map accumulator, the system can efficiently revoke users and/or their attributes, and authenticate them prior to launching any expensive search operations. Fifth, conjunctive keyword search is provided thus enabling to search for multiple keywords simultaneously, with minimal cost. Sixth, the performance analysis shows that the proposed scheme outperforms closely-related works..

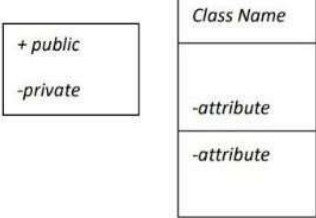
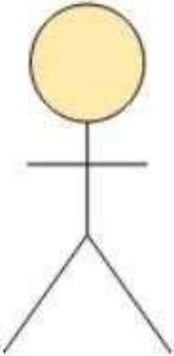
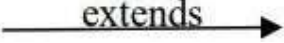
S.NO		TABLE OF CONTENTS	PAGE NO.
1		INTRODUCTION	1
	1.1	INDRODUCTION	1
	1.2	IMPLEMENTATION	3
2		SYSTEM FEASIBILITY	4
	2.1	GENERAL	5
	2.2	EXISTING SYSTEM	5
	2.3	DISADVANTAGES OF THE EXISTING SYSYTEM	5
	2.4	PROPOSED SYSTEM	6
	2.5	ADVANTAGES OF PROPOSED SYSTEM	6
3		SYSTEM ANALYSIS	7
	3.1	THE STUDY OF THE SYSYTEM	7
	3.2	INPUT AND OUTPUT REPRESENTATION	9
	3.3	INTRODUCTION TO JAVA	10
	3.4	SYSTEM REQUIREMENTS	26
	3.4.1	HARDWARE REQUIREMENTS	26
	3.4.2	SOFTWARE REQUIREMENTS	26
4		SYSTEM DESIGN	27
	4.1	SYSTEM ARCHITECTURE	27
	4.2	UML DIAGRAMS	28
	4.2.1	USE CASE DIAGRAM	32
	4.2.2	SEQUENCE DIAGRAM	34
	4.2.3	CLASS DIAGRAM	35
	4.2.4	ACTIVITY DIAGRAM	36

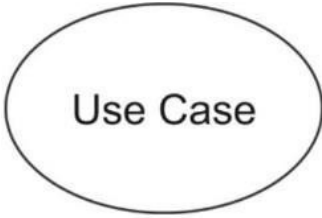
5		IMPLEMENTATION	37
	5.1	CODE	37
6		TESTING AND VALIDATION	59
	6.1	TESTING	59
	6.2	LEVELS OF TESTING	59
	6.2.1	BLACKBOX TESTING	60
	6.2.2	WHITE BOX TESTING	61
	6.3	TEST CASES	63
7		RESULTS	66
	7.1	SCREENSHOTS	66
8		CONCLUSION	68
9		FUTURE SCOPE	70
	9.2	ADVANTAGES	71
	9.3	DISADVANTAGES	73
10		REFERENCES	76

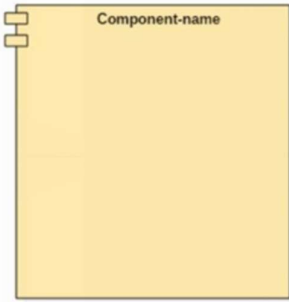
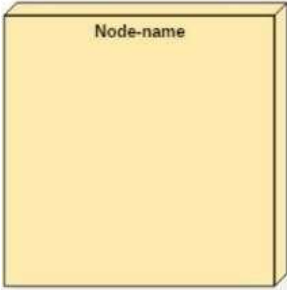
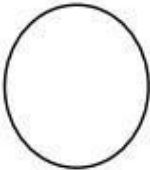
Fig. No	LIST OF FIGURES	PAGE NO.
4.2.1	System Architecture Of Project	27
4.2.1.1.1	Use Case Diagram	33
4.2.1.2.1	Sequence Diagram	34
4.2.1.3.1	Class Diagram Of User And E-Commerce Diagram	35
4.2.1.4.1	Activity Diagram	36

Screenshot. No	LIST OF SCREENSHOTS	Page. No
7.1.1	User Requests	66
7.1.2	Secret Key Verification	66
7.1.3	Matching Of Keywords	67
8.1.4	Encrypted Data	67

LIST OF SYMBOLS

SNO.	Name of Symbol	Notation	Description
1	CLASS		Represents a collection of similar entities grouped together.
2	ASSOCIATION		Associations represent static relationships between classes. Roles represent the way the two classes see each other.
3	ACTOR		It aggregates several classes into a single class.
4	RELATION (uses)	<i>Uses</i>	Used for additional process communication.
5	RELATION (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.

6	COMMUNICATION		Communication between various use cases.
7	STATE		State of the process
8	INITIAL STATE		Initial state of the object
9	FINAL STATE		Final state of the object
10	CONTROL FLOW		Represents various control flow between the states.
11	DECISION BOX		Represents decision making process from a constraint
12	USE CASE		Interact ion between the system and external environment.

13	COMPONENT		Represents physical modules which is a collection of components.
14	NODE		Represents physical modules which are a collection of components.
15	DATA PROCESS/ STATE		A circle in DFD represents a state or process which has been triggered due to some event or action.
16	EXTERNAL ENTITY		Represents external entities such as keyboard, sensors, etc
17	TRANSITION		Represents communication that occurs between processes.

18	OBJECT LIFELINE		Represents the vertical dimensions that the object communications.
19	MESSAGE	Message 	Represents the message exchanged.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Encryption-before-outsourcing is the fundamental approach to preserve data privacy and to prevent unauthorized access [2], [3]. In this approach, the data are encrypted before outsourcing to the cloud storage servers. However, the data encryption makes searching over the encrypted data a difficult task [1]. The trivial solution is to download the whole database locally, decrypt it using the corresponding key and then perform search (i.e. keyword search). Clearly, decryption of the whole database involves high computation on the local machine and is a cumbersome task. Downloading the whole database for each and every keyword search incurs heavy communication overhead on the system. Thus, this solution is not practical. The other possible solution is to let CSP decrypt the outsourced data and perform keyword search over the decrypted data. This approach violates data privacy [4], as the CSP will get to know about the plaintext data. Searchable Encryption (SE) is a preferred approach to address these issues, where the users (who use the outsourced data) delegate keyword search capabilities over encrypted data to CSP without disclosing any useful information. Notable works in this area mainly focus on single-owner and multi-user scenario, where a single owner manages his/her encrypted data and search capabilities over them [5], [6], [7], [8], [9]. Some of the single-owner and multi-user based schemes need to grant search access rights to the users via a shared secret key [5], [6], [9]; while the others need to issue search access rights in the form of trapdoors [7], [8]. The shared secret key based SE schemes [5], [6], [9] need to perform frequent key re-distribution among the authorized users to maintain a valid secret key, which is a costly and cumbersome task [10]. While the trapdoor based SE schemes [7], [8] need the Owners to be online all the time for issuing trapdoor to the authorized users. Moreover, these schemes are not suitable for multi-owner and multi user based data outsourcing platforms. The main benefit of this kind of platforms is that many Owners outsource their data and many users access those data [11]. For example, in the platforms like [12], [13], [14], many Owners share.

LITERATURE SURVEY:

Secure-Channel Free Public Key Encryption Key word Search (SCF-PKEKS)

In [28], Baek et al. highlight that Boneh et al.'s scheme needs secure-channel between a user and the CSP to transmit trapdoors, which may incur high overhead in terms of communication and processing costs. In addition, if (somehow) an adversary captures a trapdoor of a user, he/she can use the captured trapdoor for launching a replay attack. So, Baek et al. proposed an SCF-PKEKS scheme [28] to address the issue of Boneh et al.'s scheme [29]. Later, Rhee et al. proposed [44] to enhance the Baek et al.'s security model [28] for SCF PKEKS, where they allow the attacker to obtain the relation between non-challenge ciphertexts and a trapdoor. Fang et al. proposed two SCF-PKEKS schemes [30] and [45], where the former is designed without using random oracle model and the latter, is designed to prevent outsider keyword guessing attack without random oracle. Recently, Jiang et al. proposed an SCF-PKEKS scheme [27] where the users can perform keywordsearchoverthe authorized encrypted data only. However, none of the SCF-PKEKS schemes provide fine-grained search authorization over encrypted data.

Public Key Encryption Authorized Keyword Search (PKAKS)

Li et al. proposed an PKAKS scheme [46] using the concept of Hierarchical Predicate Encryption [47] for multi-owner and multi-user setting, where only authorized users can perform keyword search over the encrypted indexes. However, it incurs high communication overhead, as a user needs to contact the attribute authority to generate trapdoors for each search request. Also, the scheme is suitable to perform keyword search over structured database which contains limited keywords [24]. Sun et al. addressed the issue of Li et al.'s scheme [46] and proposed an PKAKS scheme [11] using the concept of CP-ABE to provide fine-grained search authorization functionality. A few notable PKAKS schemes based on ABE techniques have appeared to provide fine grained search authorization [24], [25], [26]. However, Sun et al.'s schemes [11], [24] are not expressive, as the access policies are constructed using boolean "AND" gates only. Also, practical usability of the two schemes is questionable, due to the heavy computational overhead on the system [26]. In addition, encrypted index size linearly increases with the total number of attributes in the system. Moreover, the schemes [11], [24] need secure-channel between a user and CSP to transmit the trapdoors which is essential to prevent replay attacks⁶ and keyword guessing attacks. Cui et al. proposed a KP-ABE based keyword search scheme [26], which uses both "AND" and "OR" in the access policies. However, in [26], a trusted authority is responsible for choosing access policy related to the keywords to be searched, i.e., the trusted third-party needs to be contacted by the users each time a fresh keyword search is requested for embedding the chosen access

policy in the trapdoor. As the trusted third-party is required to remain online, this is a major limitation of [26]. Hu et al.'s scheme [25] is expressive, and also it can update access policies at any time with the help of the CSP using proxy re-encryption and secret sharing techniques. However, like [11], [24], Hu et al.'s scheme [25] needs secure-channels between the users and CSP to transmit trapdoors. Moreover, the scheme does not provide conjunctive keyword search and user revocation

1.2 IMPLEMENTATION

MODULES:

- Data owner Module
- Data user Module

MODULES DESCRIPTION:

DATA OWNER (DO):

In the first module, we develop the Data Owner Module. Owner Will Signup and Wait for the approval from trusted authority. After authentication is successful Owner can login, and upload record and send to encryption server provider for getting encrypted data and then upload encrypted data to cloud service provider.

In this module, data owner will check the progress status of the file upload by him/her. It has large data needed to be stored and shared in cloud system. In our scheme, the entity is in charge of defining access control policy for each uploaded file. And it uploads ciphertext to CSP. After the completion, owner logout the session

USER :

In this module, we develop the User Module. User Will registries and login on the user's page. We develop the module, such that, the User will view all files which are uploaded by data owners. Owner will send request to trusted authority for attribute verification process . If attributes are matched with file attributes then decryption key is received to user mail id. User will send file to decryption server provider and received decrypted data. User can verify key and download file.

ATTRIBUTE AUTHORITY (AA):

In this module, we develop the TA Module. TA Will register and login on the user's page. We develop the module, such that he can activate or deactivate owners and verify attribute of users who requests data and send attribute key to user mailed.

VERIFICATOIN AUTHORITY (VA):

CSP stores the data for DO. It faithfully executes the operations requested by DO, while it may peek over data that DO has stored in the cloud. Data user will request data from CSP who will provide data file to user.

CLOUD SERVER (CSP):

For cloud server we will use drive hq cloud where users uploaded files will be shown in cloud server in encrypted format.

CHAPTER: 2

SYSTEM FEASIBILITY

2.1 GENERAL:

The proposed cloud architecture is shown in Figure 1. It consists of five entities, namely, Attribute Authority (AA), Verification Authority (VA), Owner, Cloud Service Provider (CSP), and Users. In Figure 1, all the dotted lines represent non secure-channels, while all the solid lines represent secure channels. AA is an independent trusted third-party entity which maintains public parameter and master secret in the system. It also issues access rights in the form of secret keys to the users based on the attributes they possess. VA is also another third-party entity which keeps secret information received from the AA. It is to be noted that VA can also be a set of designated servers under the control of the Cloud Service Provider. VA mainly verifies search access rights of the users upon Cloud Service Provider's request relying on the shared secret information with AA. Owner outsources encrypted data to the cloud storage servers. CSP is a third party entity which maintains the cloud storage servers and performs keyword search over the encrypted data based on a valid search access right of a user and Users are requesting entities that are authorized to search and access encrypted data based on their granted privileges..

2.2 EXISTING SYSTEM:

Baek et al. highlight that Boneh et al.'s scheme needs secure-channel between a user and the CSP to transmit trapdoors, which may incur high overhead in terms of communication and processing costs. In addition, if (somehow) an adversary captures a trapdoor of a user, he/she can use the captured trapdoor for launching a replay attack. So, Baek et al. proposed an SCF-PKEKS scheme to address the issue of scheme. Later, Rhee et al. proposed to enhance the Baek et al.'s security model for SCF PKEKS, where they allow the attacker to obtain the relation between non-challenge ciphertexts and a trapdoor. Fang et al. proposed two SCF-PKEKS schemes

2.3 DISADVANTAGES OF EXISTING SYSTEM:

Most of existing SE schemes assume that every user can access all the shared files. Such assumption does not hold in the cloud environment where users are actually granted different access permissions according to the access-control policy determined by data owners.

Downloading the whole database for each and every keyword search incurs heavy communication overhead on the system. Thus, this solution is not practical. The other possible solution is to let CSP decrypt the outsourced data and perform keyword search over the decrypted data. This approach violates data privacy

2.4 PROPOSED SYSTEM:

Investigate the issues of keyword search mechanisms and propose a secure and efficient scheme for authorized keyword search over encrypted data for multi-owner and multi-user cloud environment .

An expressive fine-grained authorized keyword search scheme is designed for multi-owner and multiuser scenarios using the concept of CP-ABE.

- an interactive protocol is constructed between the user and CSP to avoid overhead involved in establishing secure-channel.
- essential functionalities, like conjunctive keyword search and user revocation, are added without any significant overhead.
- the proposed scheme is proved semantically secure against chosen keyword attack and outsider's keyword guessing attack under the standard assumptions.
- the implementation of the proposed scheme show that it takes substantially less computation time to perform keyword search

2.5 ADVANTAGES OF PROPOSED SYSTEM:

This enables the cloud users, i.e. individuals/ organizations, to take the advantages offered by cloud computing such as on demand service, low investment cost, ubiquitous and flexible access.

Fast way to access data from cloud without sending request , Query request form keyword can help to save data transmission time and reduce overhead on cloud.

CHAPTER: 3

SYSTEM ANALYSIS

3.1 THE STUDY OF THE SYSTEM

- To conduct studies and analyses of an operational and technological nature, and
- To promote the exchange and development of methods and tools for operational analysis as applied to defense problems.

Logical design

The logical design of a system pertains to an abstract representation of the data flows, inputs and outputs of the system. This is often conducted via modeling, using an over-abstract (and sometimes graphical) model of the actual system. In the context of systems design are included. Logical design includes ER Diagrams i.e. Entity Relationship Diagrams

Physical design

The physical design relates to the actual input and output processes of the system. This is laid down in terms of how data is input into a system, how it is verified / authenticated, how it is processed, and how it is displayed as output. In Physical design, following requirements about the system are decided.

1. Input requirement,
2. Output requirements,
3. Storage requirements,
4. Processing Requirements,
5. System control and backup or recovery.

Put another way, the physical portion of systems design can generally be broken down into three sub-tasks:

1. User Interface Design
2. Data Design
3. Process Design

User Interface Design is concerned with how users add information to the system and with how the system presents information back to them. Data Design is concerned with how the data is represented and stored within the system. Finally, Process Design is concerned with how data moves through the system, and with how and where it is validated, secured and/or transformed as it flows into, through and out of the system. At the end of the systems design phase, documentation describing the three sub-tasks is produced and made available for use in the next phase.

Physical design, in this context, does not refer to the tangible physical design of an information system. To use an analogy, a personal computer's physical design involves input via a keyboard, processing within the CPU, and output via a monitor, printer, etc. It would not concern the actual layout of the tangible hardware, which for a PC would be a monitor, CPU, motherboard, hard drive, modems, video/graphics cards, USB slots, etc. It involves a detailed design of a user and a product database structure processor and a control processor. The H/S personal specification is developed for the proposed system.

3.2 INPUT & OUTPUT REPRESENTATION

Input Design

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

Objectives

Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

Output Design

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

- a) Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
- b) Select methods for presenting information.
- c) Create document, report, or other formats that contain information produced by the system.

3.3 INTRODUCTION TO JAVA

Java Technology

Java technology is both a programming language and a platform.

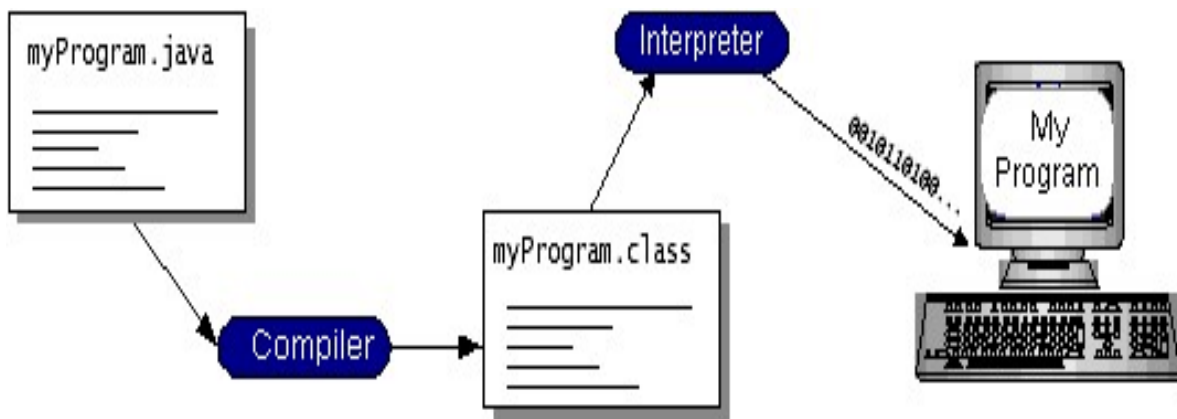
The Java Programming Language

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple

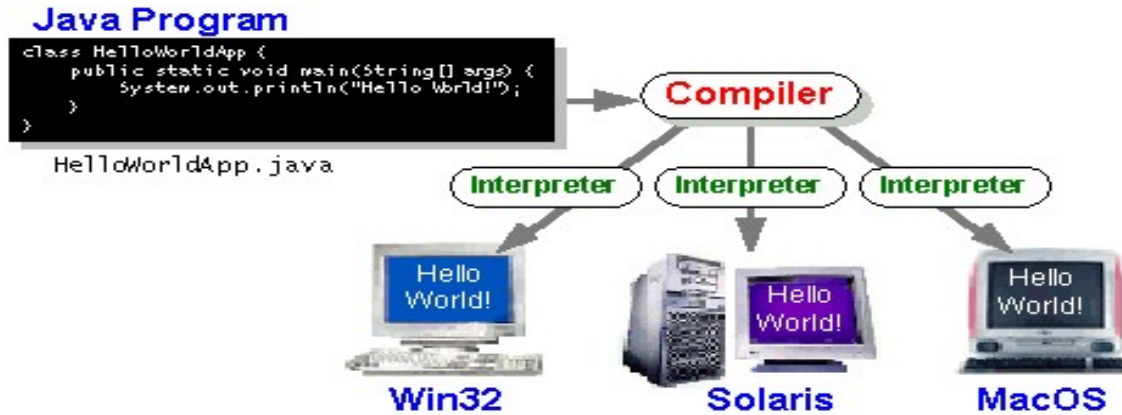
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Java byte codes—the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The following figure illustrates how this works.



You can think of Java byte codes as the machine code instructions for the *Java Virtual Machine* (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make “write once, run anywhere” possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be

run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.



The Java Platform

A platform is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

1. The Java Virtual Machine (Java VM)
2. The Java Application Programming Interface (Java API)

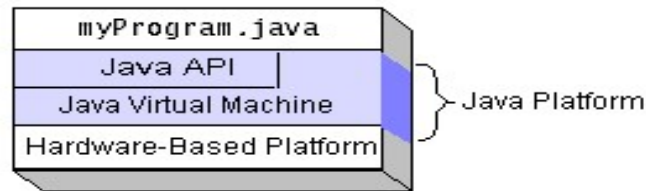
You've already been introduced to the Java VM. It's the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as packages. The next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.

Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart

compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability.



What Can Java Technology Do?

The most common types of programs written in the Java programming language are applets and applications. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

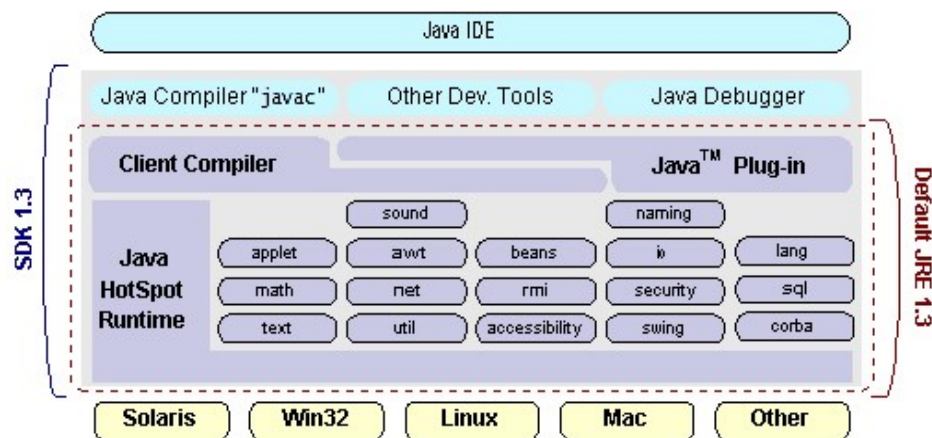
An application is a standalone program that runs directly on the Java platform. A special kind of application known as a server serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a servlet. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials:** Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets:** The set of conventions used by applets.
- **Networking:** URLs, TCP (Transmission Control Protocol), UDP (User Datagram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization:** Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.

- **Security:** Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components:** Known as JavaBeans™, can plug into existing component architectures.
- **Object serialization:** Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBC™):** Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.



How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly:** Although the Java programming language is a powerful object-oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- **Write less code:** Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- **Write better code:** The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.

- **Develop programs more quickly:** Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.
- **Avoid platform dependencies with 100% Pure Java:** You can keep your program portable by avoiding the use of libraries written in other languages. The 100% Pure Java™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.
- **Write once, run anywhere:** Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java platform.
- **Distribute software more easily:** You can upgrade applets easily from a central server. Applets take advantage of the feature of allowing new classes to be loaded “on the fly,” without recompiling the entire program.

ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a de facto standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC datasources.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named

Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC datasources. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or SQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year. JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

From a programming perspective, the beauty of ODBC is that the application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or SQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or SQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or drivers. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution. JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

JDBC Goals

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java. The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

SQL Level API

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to “generate” JDBC code and to hide many of JDBC’s complexities from the end user.

SQL Conformance

SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

JDBC must be implemental on top of common database interfaces

The JDBC SQL API must “sit” on top of other common SQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

Provide a Java interface that is consistent with the rest of the Java system

Because of Java’s acceptance in the user community thus far, the designers feel that they should not stray from the current design of the core Java system.

1. **Keep it simple**

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

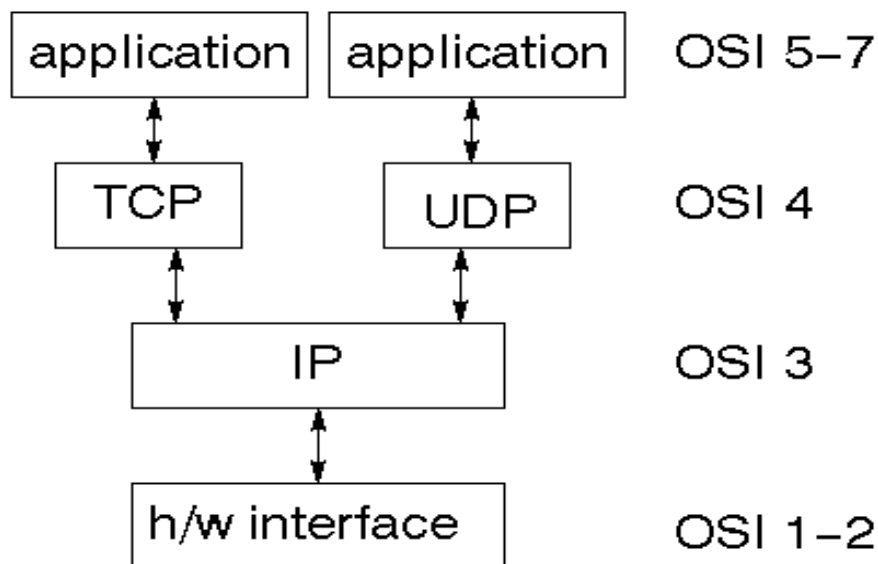
2. Use strong, static typing wherever possible

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

Networking

TCP/IP stack

The TCP/IP stack is shorter than the OSI one:



TCP is a connection-oriented protocol; UDP (User Datagram Protocol) is a connectionless protocol.

IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses. The IP layer handles routing through an Internet. It is also responsible for breaking up large datagram into smaller ones for transmission and reassembling them at the other end.

UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.

TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing. The network ID falls into various classes according to the size of the network address.

Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16 bit network addressing. Class C uses 24 bit network addressing and class D uses all 32.

Subnet address

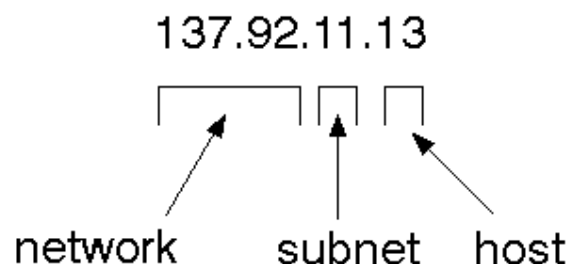
Internally, the UNIX network is divided into sub networks. Building 11 is currently on one sub network and uses 10-bit addressing, allowing 1024 different hosts.

Host address

8 bits are finally used for host addresses within our subnet. This places a limit of 256 machines that can be on the subnet.

Total address

The 32 bit address is usually written as 4 integers separated by dots.



Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

Sockets

A socket is a data structure maintained by the system to handle network connections. A socket is created using the call `socket`. It returns an integer that is like a file descriptor. In fact, under Windows, this handle can be used with `Read File` and `Write File` functions.

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int family, int type, int protocol);
```

Here "family" will be `AF_INET` for IP communications, protocol will be zero, and type will depend on whether TCP or UDP is used. Two processes wishing to communicate over a network create a socket each. These are similar to two ends of a pipe - but the actual pipe does not yet exist.

JFree Chart

JFreeChart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFreeChart's extensive feature set includes:

- A consistent and well-documented API, supporting a wide range of chart types;
- A flexible design that is easy to extend, and targets both server-side and client-side applications;

Support for many output types, including Swing components, image files (including PNG and JPEG), and vector graphics file formats (including PDF, EPS and SVG);

1. Map Visualizations

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b) income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include:

Sourcing freely redistributable vector outlines for the countries of the world, states/provinces in particular countries (USA in particular, but also other areas);

Creating an appropriate dataset interface (plus default implementation), a rendered, and integrating this with the existing XYPlot class in JFreeChart;

Testing, documenting, testing some more, documenting some more.

2. Time Series Chart Interactivity

Implement a new (to JFreeChart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding "view" rectangle that allows you to select the subset of the time series data to display in the main chart.

3. Dashboards

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFreeChart chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

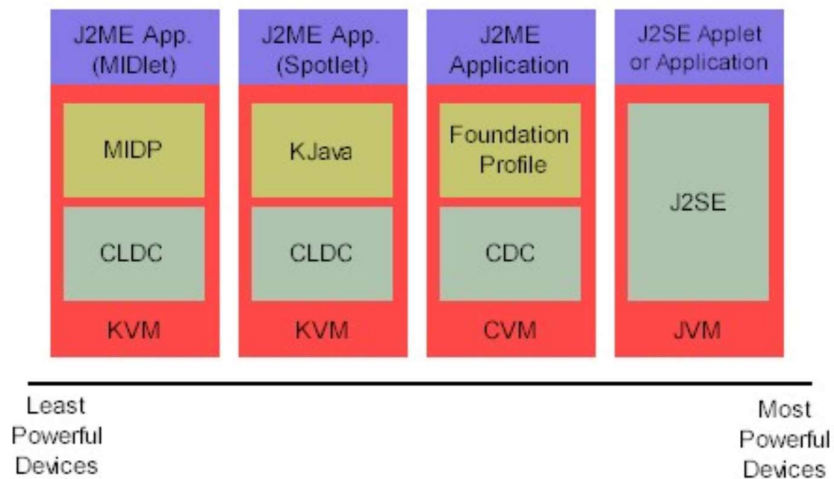
4. Property Editors

The property editor mechanism in JFreeChart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

J2ME (Java 2 Micro edition):-

Sun Microsystems defines J2ME as "a highly optimized Java run-time environment targeting a wide range of consumer products, including pagers, cellular phones, screen-phones, digital set-top boxes and car navigation systems." Announced in June 1999 at the JavaOne Developer Conference, J2ME brings the cross-platform functionality of the Java language to smaller devices, allowing mobile wireless devices to share applications. With J2ME, Sun has adapted the Java platform for consumer products that incorporate or are based on small computing devices.

1. General J2ME architecture



J2ME uses configurations and profiles to customize the Java Runtime Environment (JRE). As a complete JRE, J2ME is comprised of a configuration, which determines the JVM used, and a profile, which defines the application by adding domain-specific classes. The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. We'll discuss configurations in detail in the profile defines the application; specifically, it adds domain-specific classes to the J2ME configuration to define certain uses for devices. We'll cover profiles in depth in the The following graphic depicts the relationship between the different virtual machines, configurations, and profiles. It also draws a parallel with the J2SE API and its Java virtual machine. While the J2SE virtual machine is generally referred to as a JVM, the J2ME virtual machines, KVM and CVM, are subsets of JVM. Both KVM and CVM can be thought of as a kind of Java virtual machine - it's just that they are shrunken versions of the J2SE JVM and are specific to J2ME.

2. Developing J2ME applications

Introduction In this section, we will go over some considerations you need to keep in mind when developing applications for smaller devices. We'll take a look at the way the compiler is invoked when using J2SE to compile J2ME applications. Finally, we'll explore packaging and deployment and the role preverification plays in this process.

3. Design considerations for small devices

Developing applications for small devices requires you to keep certain strategies in mind during the design phase. It is best to strategically design an application for a small device before you begin coding. Correcting

the code because you failed to consider all of the "gotchas" before developing the application can be a painful process. Here are some design strategies to consider:

- * Keep it simple. Remove unnecessary features, possibly making those features a separate, secondary application.

- * Smaller is better. This consideration should be a "no brainer" for all developers. Smaller applications use less memory on the device and require shorter installation times. Consider packaging your Java applications as compressed Java Archive (jar) files.

- * Minimize run-time memory use. To minimize the amount of memory used at run time, use scalar types in place of object types. Also, do not depend on the garbage collector. You should manage the memory efficiently yourself by setting object references to null when you are finished with them. Another way to reduce run-time memory is to use lazy instantiation, only allocating objects on an as-needed basis. Other ways of reducing overall and peak memory use on small devices are to release resources quickly, reuse objects, and avoid exceptions.

4. Configurations overview

The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. Currently, two configurations exist for J2ME, though others may be defined in the future:

- * **Connected Limited Device Configuration (CLDC)** is used specifically with the KVM for 16-bit or 32-bit devices with limited amounts of memory. This is the configuration (and the virtual machine) used for developing small J2ME applications. Its size limitations make CLDC more interesting and challenging (from a development point of view) than CDC. CLDC is also the configuration that we will use for developing our drawing tool application. An example of a small wireless device running small applications is a Palm handheld computer.

- * **Connected Device Configuration (CDC)** is used with the C virtual machine (CVM) and is used for 32-bit architectures requiring more than 2 MB of memory. An example of such a device is a Net TV box.

5. J2ME profiles

What is a J2ME profile?

As we mentioned earlier in this tutorial, a profile defines the type of device supported. The Mobile Information Device Profile (MIDP), for example, defines classes for cellular phones. It adds domain-specific

classes to the J2ME configuration to define uses for similar devices. Two profiles have been defined for J2ME and are built upon CLDC: KJava and MIDP. Both KJava and MIDP are associated with CLDC and smaller devices. Profiles are built on top of configurations. Because profiles are specific to the size of the device (amount of memory) on which an application runs, certain profiles are associated with certain configurations. A skeleton profile upon which you can create your own profile, the Foundation Profile, is available for CDC.

Profile 1: KJava

KJava is Sun's proprietary profile and contains the KJava API. The KJava profile is built on top of the CLDC configuration. The KJava virtual machine, KVM, accepts the same byte codes and class file format as the classic J2SE virtual machine. KJava contains a Sun-specific API that runs on the Palm OS. The KJava API has a great deal in common with the J2SE Abstract Windowing Toolkit (AWT). However, because it is not a standard J2ME package, its main package is `com.sun.kjava`. We'll learn more about the KJava API later in this tutorial when we develop some sample applications.

Profile 2: MIDP

MIDP is geared toward mobile devices such as cellular phones and pagers. The MIDP, like KJava, is built upon CLDC and provides a standard run-time environment that allows new applications and services to be deployed dynamically on end user devices. MIDP is a common, industry-standard profile for mobile devices that is not dependent on a specific vendor. It is a complete and supported foundation for mobile application development. MIDP contains the following packages, the first three of which are core CLDC packages, plus three MIDP-specific packages.

- * `java.lang`
- * `java.io`
- * `java.util`
- * `javax.microedition.io`
- * `javax.microedition.lcdui`
- * `javax.microedition.midlet`
- * `javax.microedition.rms`

3.4 SYSTEM REQUIREMENTS

3.4.1 HARDWARE REQUIREMENTS:

- System : Pentium IV 2.4 GHz.

- Hard Disk : 200 GB.
- Floppy Drive : 1.44 Mb.
- Monitor : 15 VGA Colour.
- Mouse : Logitech.
- Ram : 1 GB.

3.4.2 SOFTWARE REQUIREMENTS:

- Operating system : Windows XP/7/10.
- Coding Language : Java
- Tool : Netbeans
- Database : MYSQL

CHAPTER: 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Architecture Flow:

Below architecture diagram represents mainly flow of request from the users to database through servers. In this scenario overall system is designed in three tiers separately using three layers called presentation layer, business layer, data link layer. This project was developed using 3-tier architecture.

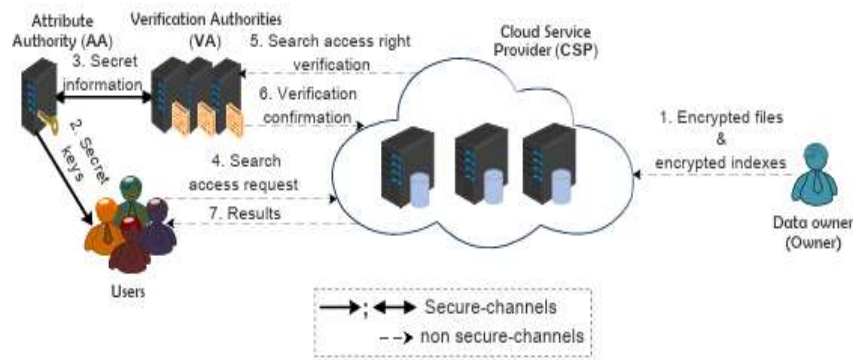


Figure 3.1: Architecture diagram

3-Tier Architecture:

The three-tier software architecture (a three layer architecture) emerged in the 1990s to overcome the limitations of the two-tier architecture. The third tier (middle tier server) is between the user interface (client) and the data management (server) components. This middle tier provides process management where business logic and rules are executed and can accommodate hundreds of users (as compared to only 100 users with the two tier architecture) by providing functions such as queuing, application execution, and database staging.

The three tier architecture is used when an effective distributed client/server design is needed that provides (when compared to the two tier) increased performance, flexibility, maintainability, reusability, and scalability, while hiding the complexity of distributed processing from the user. These characteristics have made three layer architectures a popular choice for Internet applications and net-centric information systems

Advantages of Three-Tier:

- Separates functionality from presentation.
- Clear separation – better understanding.
- Changes limited to well define components.

- Can be running on WWW.
- Effective network performance.

SYSTEM DESIGN

System Design Introduction:

The System Design Document describes the system requirements, operating environment, system and subsystem architecture, files and database design, input formats, output layouts, human-machine interfaces, detailed design, processing logic, and external interfaces.

4.2 UML DIAGRAMS

Unified Modeling Language (UML) is a comprehensive visual modeling language used for specifying, visualizing, constructing, and documenting the artifacts of software systems. It emerged in the mid-1990s as a unification of several object-oriented modeling approaches, aiming to standardize the notation and semantics of software modeling. Let's delve into more detail about various aspects of UML:

1. Types of Diagrams:

UML provides several types of diagrams, each serving a specific purpose:

Structural Diagrams: These diagrams depict the static structure of a system, showing the components that make up the system and how they relate to each other. Examples include Class Diagrams, Object Diagrams, Component Diagrams, and Deployment Diagrams.

Behavioral Diagrams: Behavioral diagrams illustrate the dynamic behavior of a system, focusing on the interactions and collaborations between the system's components. Key behavioral diagrams include Use Case Diagrams, Sequence Diagrams, Collaboration Diagrams (deprecated in UML 2.x and replaced by Communication Diagrams), Statechart Diagrams, Activity Diagrams, and Timing Diagrams.

Interaction Diagrams: These diagrams emphasize the flow of messages and interactions between objects or components in a system. Sequence Diagrams and Communication Diagrams fall into this category.

Deployment Diagrams: Deployment diagrams show the physical deployment of artifacts on hardware nodes, depicting how software components are distributed across hardware nodes.

2. Modeling Concepts:

Classes and Objects: UML class diagrams represent the structure of a system by showing classes, their attributes, methods, and relationships with other classes.

Relationships: UML supports various types of relationships between classes, such as association, aggregation, composition, inheritance, and dependency.

Behavioral Elements: UML behavior diagrams capture the dynamic aspects of a system, such as the flow of control and data during system execution.

Use Cases: Use case diagrams depict the functional requirements of a system by showing the interactions between actors (users or external systems) and the system itself.

3. Tool Support:

There are numerous tools available for creating, editing, and analyzing UML diagrams. These tools range from simple drawing tools to sophisticated modeling environments that offer features like code generation, reverse engineering, model validation, and collaboration support.

4. Adaptability:

UML is flexible and can be adapted to various development methodologies, including but not limited to object-oriented, component-based, and service-oriented approaches. It can also be customized to suit the specific needs and conventions of different domains or organizations.

5. Standardization and Evolution:

UML is maintained by the Object Management Group (OMG), which regularly releases new versions of the language to incorporate improvements and address emerging needs in software engineering. While UML has

evolved over time, the core concepts and principles have remained largely consistent, ensuring compatibility and continuity across different versions.

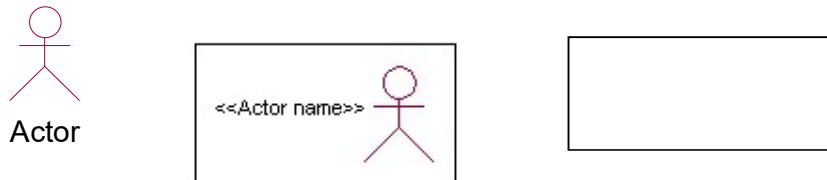
In summary, UML serves as a powerful tool for software engineers and other stakeholders involved in the software development process, providing a standardized and expressive language for modeling software systems from various perspectives.

Global Use Case Diagrams:

Identification of actors:

Actor: Actor represents the role a user plays with respect to the system. An actor interacts with, but has no control over the use cases.

Graphical representation:



An actor is someone or something that:

Interacts with or uses the system.

- Provides input to and receives information from the system.
- Is external to the system and has no control over the use cases.

Actors are discovered by examining:

- Who directly uses the system?
- Who is responsible for maintaining the system?
- External hardware used by the system.
- Other systems that need to interact with the system.

Questions to identify actors:

- Who is using the system? Or, who is affected by the system? Or, which groups need help from the system to perform a task?

- Who affects the system? Or, which user groups are needed by the system to perform its functions?

These functions can be both main functions and secondary functions such as administration.

- Which external hardware or systems (if any) use the system to perform tasks?
- What problems does this application solve (that is, for whom)?
- And, finally, how do users use the system (use case)? What are they doing with the system?

The actors identified in this system are:

- a. System Administrator
- b. Customer
- c. Customer Care

Identification of usecases:

Usecase: A use case can be described as a specific way of using the system from a user's (actor's) perspective.

Graphical representation:



A more detailed description might characterize a use case as:

- Pattern of behavior the system exhibits
- Interacts with or uses the system.
- Provides input to and receives information from the system.
- Is external to the system and has no control over the use cases.

Actors are discovered by examining:

- Who directly uses the system?
- Who is responsible for maintaining the system?
- External hardware used by the system.
- Other systems that need to interact with the system.

Questions to identify actors:

- Who is using the system? Or, who is affected by the system? Or, which groups need help from the system to perform a task?
- Who affects the system? Or, which user groups are needed by the system to perform its functions?

These functions can be both main functions and secondary functions such as administration.

- Which external hardware or systems (if any) use the system to perform tasks?
- What problems does this application solve (that is, for whom)?

- And, finally, how do users use the system (use case)? What are they doing with the system?

The actors identified in this system are:

- d. System Administrator
- e. Customer
- f. Customer Care

Identification of usecases:

Usecase: A use case can be described as a specific way of using the system from a user's (actor's) perspective.

Graphical representation:



A more detailed description might characterize a use case as:

- Pattern of behavior the system exhibits
- A sequence of related transactions performed by an actor and the system
- Delivering something of value to the actor

Use cases provide a means to:

- capture system requirements
- communicate with the end users and domain experts
- test the system

Use cases are best discovered by examining the actors and defining what the actor will be able to do with the system.

Guide lines for identifying use cases:

- For each actor, find the tasks and functions that the actor should be able to perform or that the system needs the actor to perform. The use case should represent a course of events that leads to clear goal
- Name the use cases.
- Describe the use cases briefly by applying terms with which the user is familiar.

This makes the description less ambiguous

Questions to identify use cases:

- What are the tasks of each actor?

- Will any actor create, store, change, remove or read information in the system?
- What use case will store, change, remove or read this information?
- Will any actor need to inform the system about sudden external changes?
- Does any actor need to inform about certain occurrences in the system?
- What usecases will support and maintains the system?

Flow of Events

A flow of events is a sequence of transactions (or events) performed by the system. They typically contain very detailed information, written in terms of what the system should do, not how the system accomplishes the task. Flow of events are created as separate files or documents in your favorite text editor and then attached or linked to a use case using the Files tab of a model element.

A flow of events should include:

- When and how the use case starts and ends
- Use case/actor interactions
- Data needed by the use case
- Normal sequence of events for the use case
- Alternate or exceptional flows

4.2.1 USECASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

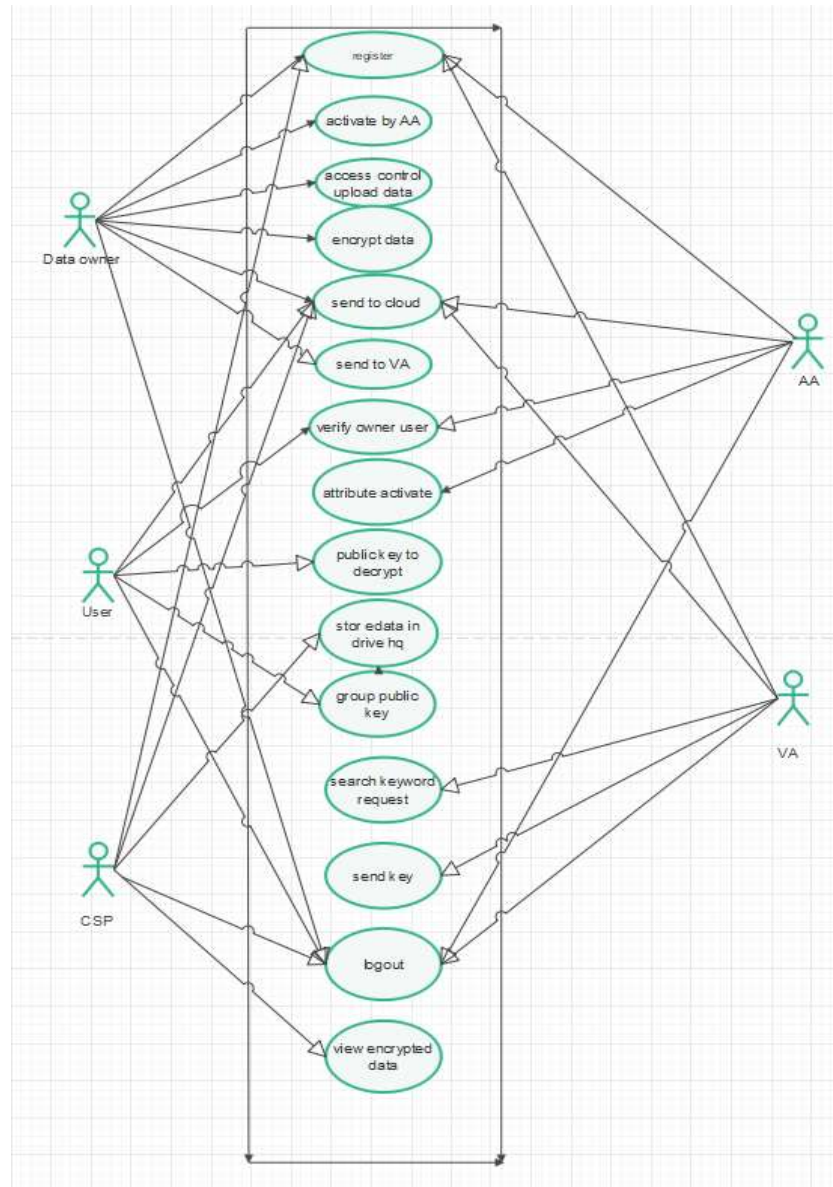


Figure 4.2.1 Use Case Diagram

4.2.2 SEQUENCE DIAGRAMS:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

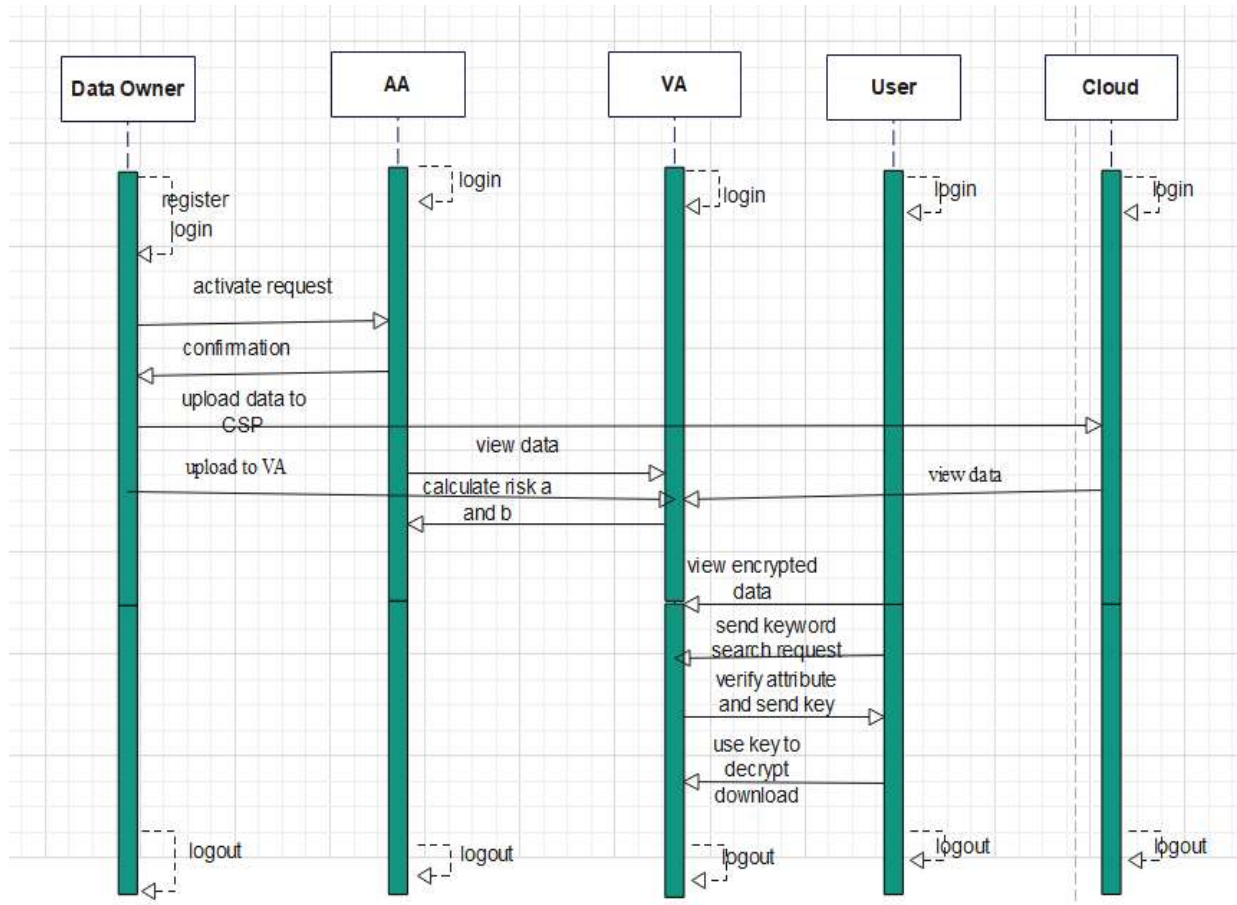


Figure 4.2.2: Sequence diagram

4.2.3. CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

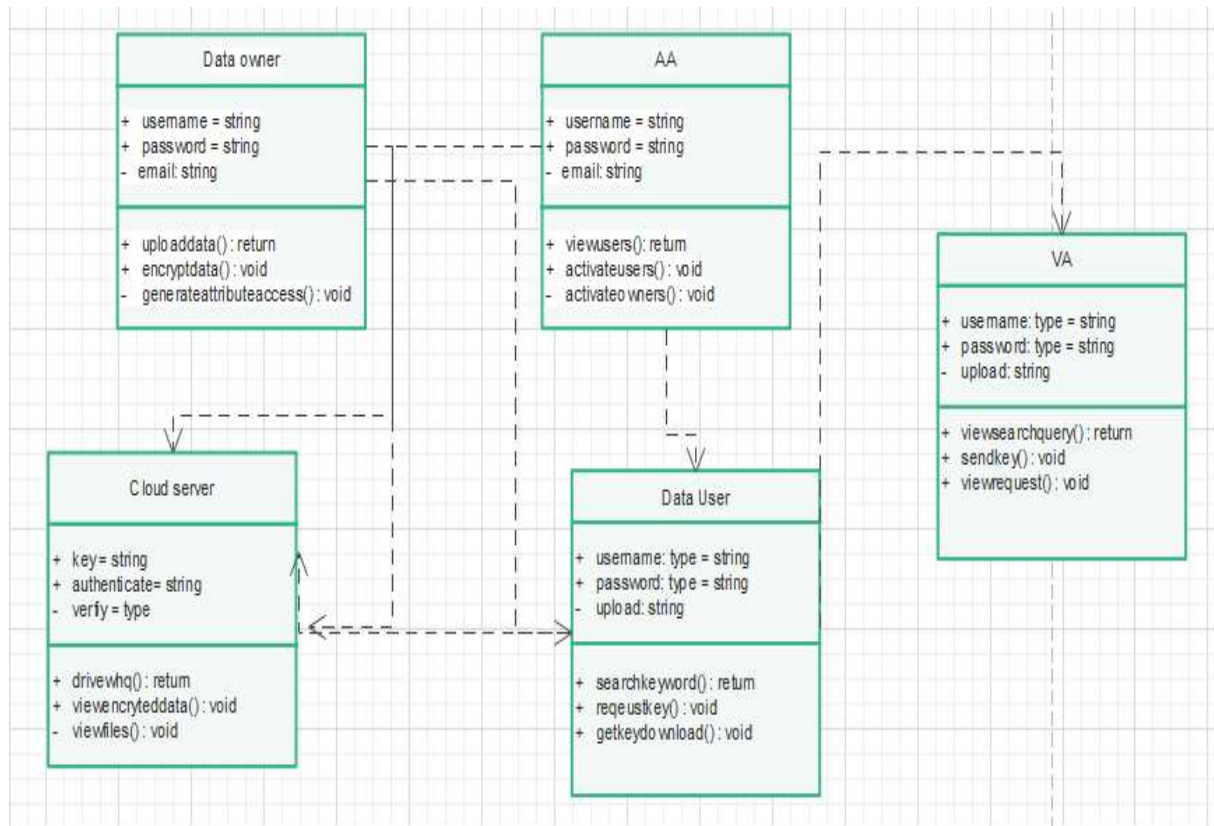


Figure 4.2.3: Class Diagram

4.2.4 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

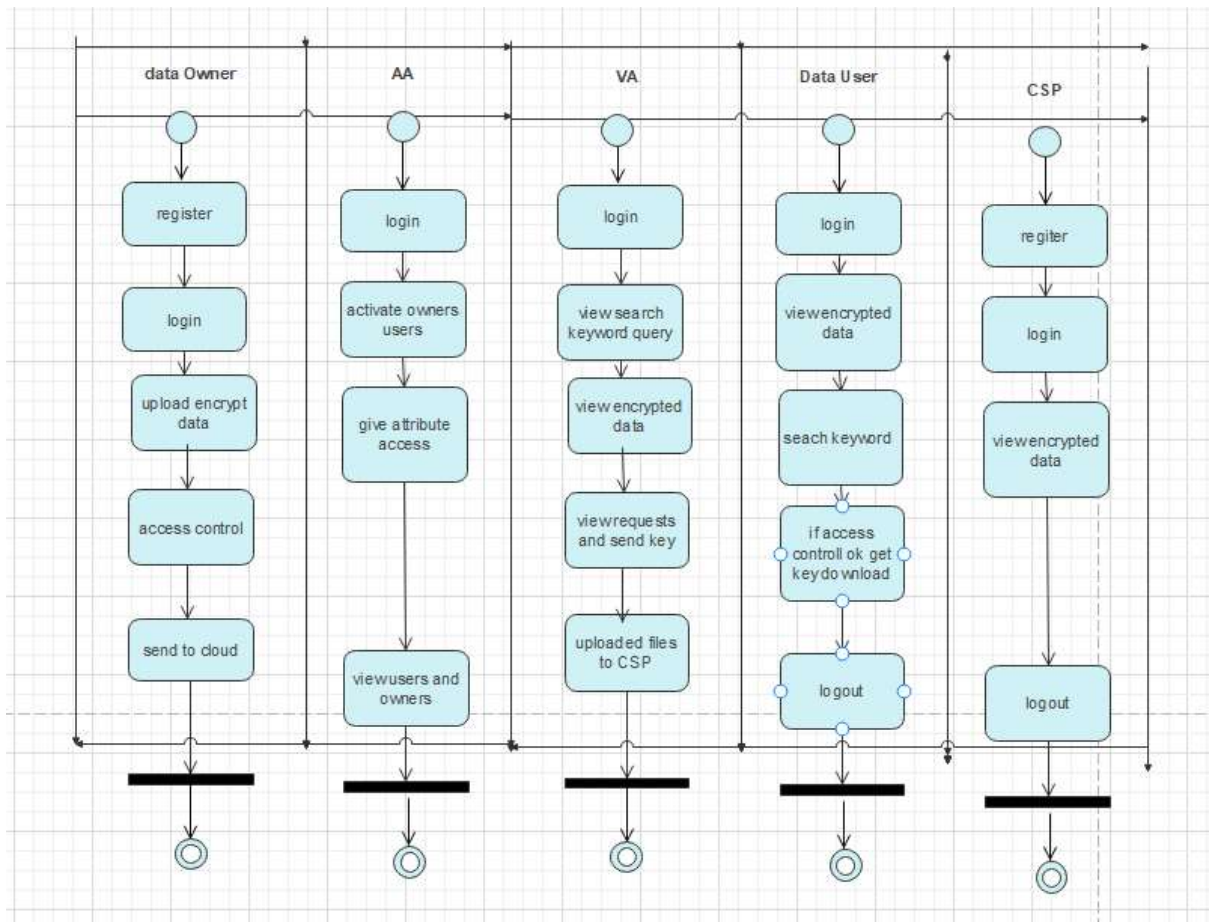


Figure 4.2.4: Activity Diagram

CHAPTER 5

IMPLEMENTATION

5.1 CODE

Databaseconnection

```
package databaseconnection;
import java.sql.*;

public class databasecon
{
    static Connection co;
    public static Connection getconnection()
    {
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            co
DriverManager.getConnection("jdbc:mysql://localhost:3306/securedata","root","root");
        }
        catch(Exception e)
        {
            System.out.println("Database Error"+e);
        }
        return co;
    }
}
```

Decryption

```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package action;

/**
 *
 * @author java2
 */
import com.sun.org.apache.xerces.internal.impl.dv.util.Base64;
import java.io.ByteArrayOutputStream;
```

```

import java.io.FileInputStream;
import java.io.FileWriter;
import java.util.Scanner;

import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.JOptionPane;
import sun.misc.BASE64Decoder;
import sun.misc.BASE64Encoder;

public class decryption {
//public static void main(String args[])
//{
//    Scanner s=new Scanner(System.in);
//    System.out.println("Enter encrypted Text and key");
//    String text=s.next();
//    String key=s.next();
//    new decryption().decrypt(text,key);
//}

    public String decrypt(String txt, String skey) {
        String decryptedtext = null;
        try {

            //converting string to secretkey
            byte[] bs = Base64.decode(skey);
            SecretKey sec = new SecretKeySpec(bs, "AES");
            System.out.println("converted string to seretkey:" + sec);

            System.out.println("secret key:" + sec);

            Cipher aesCipher = Cipher.getInstance("AES");//getting AES instance
            aesCipher.init(Cipher.ENCRYPT_MODE, sec);//initiating ciper encryption using secretkey

            byte[] byteCipherText = new BASE64Decoder().decodeBuffer(txt); //encrypting data

            // System.out.println("ciper text:"+byteCipherText);
            aesCipher.init(Cipher.DECRYPT_MODE, sec, aesCipher.getParameters());//initiating ciper
decryption

            byte[] byteDecryptedText = aesCipher.doFinal(byteCipherText);
            decryptedtext = new String(byteDecryptedText);

            System.out.println("Decrypted Text:" + decryptedtext);
        } catch (Exception e) {
            System.out.println(e);
        }
        return decryptedtext;
    }
}

```

```

    }

    String decrypt(String str, SecretKey sec) {
        throw new UnsupportedOperationException("Not supported yet."); //To change body of generated
methods, choose Tools | Templates.
    }

}

```

Download

```

<%@page import="java.sql.*"%>
<%@page import="Distributed.Dbconnection"%>
<%@ page session="true" %>
<%
String username = session.getAttribute("user").toString();
String task = request.getParameter("task");
String location = request.getParameter("location");
String status = null;
String person = null;

try{

Connection con=Dbconnection.getConnection();
PreparedStatement ps=con.prepareStatement("insert into task values(?,?,?,?)");

ps.setString(1,username);
ps.setString(2,task);
ps.setString(3,location);

ps.setString(4,person);
ps.setString(5,status);
int i=ps.executeUpdate();
if(i>0)
{
response.sendRedirect("owner_upload.jsp?msg=Registered");
}
else{
response.sendRedirect("ownerreg.jsp?msg1=Failed");
}
}%>
<%
}

catch(Exception e)
{

    out.println(e);
}

```

```
}  
%>
```

View task

```
<%@page import="java.sql.ResultSet"%>  
<%@page import="java.sql.PreparedStatement"%>  
<%@page import="Distributed.Dbconnection"%>  
<%@page import="java.sql.Connection"%>  
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
<head>  
<title>Distributed Time-Sensitive Task Selection in Mobile Crowd Sensing</title>  
<meta http-equiv="Content-type" content="text/html; charset=utf-8" />  
<link rel="stylesheet" href="css/style.css" type="text/css" media="all" />  
<script type="text/javascript" src="js/jquery-1.4.2.min.js"></script>  
<script type="text/javascript" src="js/jquery.jcarousel.js"></script>  
<script src="js/cufon-yui.js" type="text/javascript" charset="utf-8"></script>  
<script src="js/Chaparral_Pro.font.js" type="text/javascript" charset="utf-8"></script>  
<script type="text/javascript" src="js/jquery-func.js"></script>  
<link rel="shortcut icon" type="image/x-icon" href="css/images/favicon.ico" />  
</head>  
<%  
if(request.getParameter("mg")!=null){%>  
  
    <script>alert('Login Sucessfully..!')</script>  
}  
  
<%}  
if(request.getParameter("msg1")!=null){%>  
  
    <script>alert('Owner Registration Failed..!')</script>  
}  
<%  
}  
%>  
<%  
if(request.getParameter("m3")!=null){%>  
  
    <script>alert('username already exists')</script>  
}  
  
<%}  
if(request.getParameter(" ")!=null){%>  
  
    <script>alert('Owner Registration Failed..!')</script>  
}  
<%
```

```

}
%>
<body>
<!-- START PAGE SOURCE -->
<div id="header">
  <br>
  <div class="shell">
    <h1>Distributed Time-Sensitive Task Selection in Mobile Crowd Sensing</h1>
    <div class="search">

  </div>
</div>
</div>
</div>
<div id="navigation">
  <div class="shell">
    <ul>
      <li><a href="ownerhome.jsp">HOME</a></li>
      <li><a href="owner_upload.jsp">Upload Task</a></li>
      <li><a class="active" href="owner_viewtask.jsp">View Task</a></li>
      <li><a href="owner_assign_reward.jsp">Assign Reward</a></li>
      <li><a href="owner_download.jsp">Delivery Status</a></li>
      <li><a href="logout.jsp">Logout</a></li>
    </ul>
  </div>
</div>
<div id="featured">
  <div class="shell">
    <div class="slider-carousel">
      <ul>
        <li>
          <div class="info">
            <p> We
              provide a new efficient RDPC protocol based on homomorphic hash function. The new scheme is
              provably secure against forgery attack, replace attack and replay attack based on a typical security model. To
              support data dynamics, an operation record table (ORT) is introduced to track operations on file blocks. We
              further give a new optimized implementation for the ORT which makes the cost of accessing ORT nearly
              constant. Moreover, we make the comprehensive performance analysis which shows that our scheme has
              advantages in computation and communication costs. Prototype implementation and experiments exhibit that
              the scheme is feasible for real applications. </p>
            </div>
            <div class="image"> <a href="#"></a> </div>
            <div class="cl">&nbsp;</div>
          </li>
          <li>
            <div class="info">
            </div>
            <div class="image"> <a href="#"></a> </div>
            <div class="cl">&nbsp;</div>
          </li>

```

```

</ul>
</div>
</div>
</div>
<%

String username = session.getAttribute("user").toString();
String task = request.getParameter("task");
String location = request.getParameter("location");

Connection con = null;
con = Dbconnection.getConnection();
PreparedStatement pst=con.prepareStatement("select * from taskhandler where location =
"+location+" ");
ResultSet rs=pst.executeQuery();
%>
<div id="main">
<div class="shell">
<div id="main-boxes">

<center>
<p align="justify">
<p><font color="black" size="5"> Assign Task </font></p><br/>

<form name="myform" autocomplete="off" action="owner_viewtask2.jsp" method="post">
onsubmit="return validateform()">
<table align="center" width="321">
<tr>
<td width="191" height="43"><font color="black">User Name </td>
<td width="218"><input autocomplete="off" value="<%=username%>" name="username" required=""
placeholder="User Name" /></td>
</tr>
<tr>
<td height="43"><font color="black">Task </td>
<td width="218"><input type="text" value="<%=task%>" name="task" required=""
placeholder="Password" /></td>
</tr>
<tr>
<td height="43"><font color="black"> Location</td>
<td><input name="location" autocomplete="off" value="<%=location%>" required=""
placeholder="Email ID"/></td>
</tr>

<tr>
<td height="43"><font color="black">Select Task Handler</td>
<td><select name="th" style="width:170px;" required="">
<option>--Select--</option>
<%
while(rs.next()){

```

```

%>

<option><%=rs.getString("username")%></option>

<%
}
%>
</select></td>
</tr>
<tr>
<td height="43" rowspan="3">
<p>&nbsp;</p></td>
<td align="left" valign="middle"> <p>&nbsp;</p>
<p>
<input name="submit" type="submit" value="Assign" />
</p>
<div align="right">
</div></td>
</tr>
</table>
</form>
</p>

</center>

</div>

<div class="cl">&nbsp;</div>
</div>
</div>
<div class="footer">
<div class="shell">
<p class="rf"></a></p>
<div style="clear:both;"></div>
</div>
</div>
<script type="text/javascript">pageLoaded();</script>
<!-- END PAGE SOURCE -->
</body>
</html>

```

Task handler:

```

<%@page import="java.util.Random"%>
<%@page import="java.sql.*"%>
<%@page import="Distributed.Dbconnection"%>
<%@ page session="true" %>
<%@page import="Distributed.Mail"%>

```

```

<%
String username = request.getParameter("username");
String password = request.getParameter("password");

try{

Connection con=Dbconnection.getConnection();
Random s = new Random();
int otp = s.nextInt(10000 - 5000) + 25000 ;

Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from taskhandler where email= '"+username+"' and
password='"+password+"'");
if(rs.next())
{

String user = rs.getString("username");
session.setAttribute("user",user);
String email = rs.getString("email");
session.setAttribute("email",email);
response.sendRedirect("taskhandlerhome.jsp?m1");
}
else
{
response.sendRedirect("taskhandler.jsp?msg1=LoginFail");
}
}
catch(Exception e)
{
System.out.println("Error in emplogact"+e.getMessage());
}
}%>

```

SAMPLE CODE:

```

!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Online Collaborative Data Caching using edge computing</title>

<meta http-equiv="Content-type" content="text/html; charset=utf-8" />

<link rel="stylesheet" href="css/style.css" type="text/css" media="all" />

<script type="text/javascript" src="js/jquery-1.4.2.min.js"></script>

<script type="text/javascript" src="js/jquery.jcarousel.js"></script>

```

```
<script src="js/cufon-yui.js" type="text/javascript" charset="utf-8"></script>
<script src="js/Chaparral_Pro.font.js" type="text/javascript" charset="utf-8"></script>
<script type="text/javascript" src="js/jquery-func.js"></script>
<link rel="shortcut icon" type="image/x-icon" href="css/images/favicon.ico" />
</head>
<body>
<!-- START PAGE SOURCE -->
<div id="header">
<br>
<div class="shell">
<h1> COLLABORATIVE DATA CACHING </h1>
<div class="search">
</div>
</div>
</div>
<div id="navigation">
<div class="shell">
<ul>
<li><a class="active" href="index.html">HOME</a></li>
<li><a href="mobilesource.jsp"> MOBILE SOURCE</a>
</li><li><a href="eserver.jsp">EDGE SERVER</a></li>
<li><a href="cloud.jsp">CLOUD SERVER</a></li>
<li><a href="mobiledestination.jsp">MOBILE DESTINATION</a></li>
</ul>
</div>
</div>
</div>
<div id="featured">
<div class="shell">
<div class="slider-carousel">
```


<div class="info">

<p style="font-size: 20px;">Collaborative data caching is a testament to the fact that in the world of distributed computing, cooperation and coordination can yield significant benefits and Efficiently Managing Frequently Accessed Data.</p>

</div>

<div class="image">

</div>

<div class="cl"> </div>

<div class="info">

<p style="font-size: 20px;">Collaborative data caching in edge computing is the key to faster, more efficient web page delivery by enabling distributed servers to work together to store and serve frequently accessed content closer to the end-users. </p>

</div>

<div class="image">

</div>

<div class="cl"> </div>

</div>

</div>

```

</div>
<div id="main">
<div class="shell">
<div id="main-boxes">
<br><br><br><br><br>
<br><br><br>
<center><p><font size="5" color="black">Welcome to Develop a Secure and Trust Based Key
Management Protocol for Cloud Computing Environments</font></p><br></center>

</div>
<br>
<div class="cl">&nbsp;</div>
</div>
</div>
<div class="footer">
<div class="shell">
<p class="rf"></a></p>
<div style="clear:both;"></div>
</div>
</div>
<script type="text/javascript">pageLoaded();</script>
<!-- END PAGE SOURCE -->
</body>
</html>

```

Database connection:

```

package databaseconnection; import java.sql.*;

public class databasecon
{
static Connection co;

```

```

public static Connection getConnection()
{
    try
    {
        Class.forName("com.mysql.jdbc.Driver"); co =
        DriverManager.getConnection("jdbc:mysql://localhost:3306/securedata","root","root");
    }
    catch(Exception e)
    {
        System.out.println("Database Error"+e);
    }
    return co;
}
}

```

Decryption:

```

package action;

/**
 *
 * @author java2
 */

import com.sun.org.apache.xerces.internal.impl.dv.util.Base64; import java.io.ByteArrayOutputStream;
import java.io.FileInputStream; import java.io.FileWriter; import java.util.Scanner;

import javax.crypto.Cipher;
import javax.crypto.KeyGenerator; import javax.crypto.SecretKey;

import javax.crypto.spec.SecretKeySpec; import javax.swing.JOptionPane;

import sun.misc.BASE64Decoder; import sun.misc.BASE64Encoder;

public class decryption {

```

```

//public static void main(String args[])
//{
//    Scanner s=new Scanner(System.in);
//    System.out.println("Enter encrypted Text and key");
//    String text=s.next();
//    String key=s.next();
//new decryption().decrypt(text,key);
//}
public String decrypt(String txt, String skey) { String decryptedtext = null;

try {
//converting string to secretkey byte[] bs = Base64.decode(skey);

    SecretKey sec = new SecretKeySpec(bs, "AES"); System.out.println("converted string to seretkey:" + sec);
System.out.println("secret key:" + sec);

    Cipher aesCipher = Cipher.getInstance("AES");//getting AES instance
aesCipher.init(Cipher.ENCRYPT_MODE, sec);//initiating cipher encryption

    using secretkey

    byte[] byteCipherText = new BASE64Decoder().decodeBuffer(txt); //encrypting

    data

        // System.out.println("cipher text:"+byteCipherText);
        aesCipher.init(Cipher.DECRYPT_MODE, sec,

aesCipher.getParameters());//initiating cipher decryption

    byte[] byteDecryptedText = aesCipher.doFinal(byteCipherText); decryptedtext = new
String(byteDecryptedText);

    System.out.println("Decrypted Text:" + decryptedtext);
} catch (Exception e) { System.out.println(e);
}

return decr

```

```

public class decryption {
//public static void main(String args[])
//{
//
//    Scanner s=new Scanner(System.in);
//    System.out.println("Enter encrypted Text and key");
//    String text=s.next();
//    String key=s.next();
//new decryption().decrypt(text,key);
//}
public String decrypt(String txt, String skey) { String decryptedtext = null;

try {
//converting string to secretkey byte[] bs = Base64.decode(skey);

    SecretKey sec = new SecretKeySpec(bs, "AES"); System.out.println("converted string to seretkey:" + sec);
System.out.println("secret key:" + sec);

    Cipher aesCipher = Cipher.getInstance("AES");//getting AES instance
aesCipher.init(Cipher.ENCRYPT_MODE, sec);//initiating cipher encryption

using secretkey

byte[] byteCipherText = new BASE64Decoder().decodeBuffer(txt); //encrypting

data

        // System.out.println("cipher text:"+byteCipherText);
        aesCipher.init(Cipher.DECRYPT_MODE, sec,

aesCipher.getParameters());//initiating cipher decryption

byte[] byteDecryptedText = aesCipher.doFinal(byteCipherText); decryptedtext = new
String(byteDecryptedText);

System.out.println("Decrypted Text:" + decryptedtext);
} catch (Exception e) { System.out.println(e);
}

return decr

```

```
<div id="featured">
```

```
<div class="shell">
```

```
<div class="slider-carousel">
```

```
<ul>
```

```
<li>
```

```
<div class="info">
```

```
<p style="font-size: 20px;">Collaborative data caching is a testament to the fact that in the world of distributed computing, cooperation and coordination can yield significant benefits and Efficiently Managing Frequently Accessed Data.</p>
```

```
</div>
```

```
<div class="image"> <a href="#"></a>
```

```
</div>
```

```
<div class="cl">&nbsp;</div>
```

```
</li>
```

```
<li>
```

```
<div class="info">
```

```
<p style="font-size: 20px;">Collaborative data caching in edge computing is the key to faster, more efficient web page delivery by enabling distributed servers to work together to store and serve frequently accessed content closer to the end-users. </p>
```

```
</div>
```

```
<div class="image"> <a href="#"></a>
```

```
</div>
```

```
<div class="cl">&nbsp;</div>
```

```
</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
String decrypt(String str, SecretKey sec) {
    throw new UnsupportedOperationException("Not supported yet."); //To change body of generated
methods, choose Tools | Templates.
}
}
```

Encryption:

```
*/

package collaborative;
import com.sun.org.apache.xerces.internal.impl.dv.util.Base64; import java.io.ByteArrayOutputStream;
import java.io.FileInputStream; import java.io.FileWriter; import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator; import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec; import javax.swing.JOptionPane;
import sun.misc.BASE64Encoder; public class encryption

public String encrypt(String text, SecretKey secretkey) {

String plainData = null, cipherText = null; try {

plainData = text;
Cipher aesCipher = Cipher.getInstance("AES");//getting AES instance
aesCipher.init(Cipher.ENCRYPT_MODE, secretkey);//initiating cipher
encryption using secretkey
data byte[]plainData.getBytes()byte[] byteCipherText =
byteDataToEncipher.doFinal(byteDataToEncrypt);//encrypting
cipherText = new BASE64Encoder().encode(byteCipherText);//converting encrypted data to string

System.out.println("\n Given text : " + plainData + " \n Cipher Data : " + cipherText);
```

```

    } catch (Exception e) { System.out.println(e);
    }

    return cipherText;
}
}

```

[Download:](#)

```

<%@page import="java.sql.*"%>
<%@page import="Distributed.Dbconnection"%>
<%@ page session="true" %>
<%

String username = session.getAttribute("user").toString(); String task = request.getParameter("task");

String location = request.getParameter("location"); String status = null;

String person = null; try{

Connection con=Dbconnection.getConnection();

PreparedStatement ps=con.prepareStatement("insert into task values(?,?,?,?)") ps.setString(1,username);

ps.setString(2,task); ps.setString(3,location);

ps.setString(4,person); ps.setString(5,status);

int i=ps.executeUpdate(); if(i>0)

{

response.sendRedirect("owner_upload.jsp?msg=Registered");

}

else{ response.sendRedirect("ownerreg.jsp?msg1=Failed");

}

}%>

<%

```

```
catch(Exception e)
```

```
{
```

```
out.println(e);
```

```
}
```

```
%>
```

```
package collaborative;
```

```
import java.io.File;
```

```
import java.io.FileInputStream;
```

```
import org.apache.commons.net.ftp.FTPClient;
```

```
/*
```

```
* To change this license header, choose License Headers in Project Properties.
```

```
* To change this template file, choose Tools | Templates
```

```
* and open the template in the editor.
```

```
*/
```

```
/**
```

```
*
```

```
* @author java2
```

```
*/
```

```
public class Ftpcon {
```

```
FTPClient client = new FTPClient(); FileInputStream fis = null;
```

```
boolean status;
```

```
public boolean upload(File file,String fname) { try {  
  
    client.connect("ftp.drivehq.com"); client.login("shival8", "shivagmail@8");  
client.enterLocalPassiveMode();  
  
    fis = new FileInputStream(file);  
  
  
    status = client.storeFile(" /cloud/" + fname, fis);  
    client.logout(); fis.close();  
  
} catch (Exception e) { System.out.println(e);  
  
}  
  
if (status) { System.out.println("success"); return true;  
  
} else {  
    System.out.println("failed"); return false;  
  
}  
  
}  
  
}
```

CHAPTER: 6

TESTING & VALIDATION

6.1 TESTING:

Testing is the debugging program is one of the most critical aspects of the computer programming triggers, without programming that works, the system would never produce an output of which it was designed. Testing is best performed when user development is asked to assist in identifying all errors and bugs. The sample data are used for testing. It is not quantity but quality of the data used the matters of testing. Testing is aimed at ensuring that the system was accurately an efficiently before live operation commands.

Testing objectives:

The main objective of testing is to uncover a host of errors, systematically and with minimum effort and time. Stating formally, we can say, testing is a process of executing a program with intent of finding an error.

1. A successful test is one that uncovers an as yet undiscovered error.
2. A good test case is one that has probability of finding an error, if it exists.
3. The test is inadequate to detect possibly present errors.
4. The software more or less confirms to the quality and reliable standards.

6.2 LEVELS OF TESTING:

Code testing:

This examines the logic of the program. For example, the logic for updating various sample data and with the sample files and directories were tested and verified.

Specification Testing:

Executing this specification starting what the program should do and how it should performed under various conditions. Test cases for various situation and combination of conditions in all the modules are tested.

Unit testing:

In the unit testing we test each module individually and integrate with the overall system. Unit testing focuses verification efforts on the smallest unit of software design in the module. This is also known as module testing. The module of the system is tested separately. This testing is carried out during programming stage itself. In the testing step each module is found to work satisfactorily as regard to expected output from the module. There are some validation checks for fields also. For example the validation check is done for varying the user input given by the user which validity of the data entered. It is very easy to find error debut the system.

Each Module can be tested using the following two Strategies:

1. Black Box Testing
2. White Box Testing

6.2.1 BLACK BOX TESTING

What is Black Box Testing?

Black box testing is a software testing techniques in which functionality of the software under test (SUT) is tested without looking at the internal code structure, implementation details and knowledge of internal paths of the software. This type of testing is based entirely on the software requirements and specifications.

In Black Box Testing we just focus on inputs and output of the software system without bothering about internal knowledge of the software program.



The above Black Box can be any software system you want to test. For example : an operating system like Windows, a website like Google ,a database like Oracle or even your own custom application. Under Black Box Testing , you can test these applications by just focusing on the inputs and outputs without knowing their internal code implementation.

Black box testing - Steps

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially requirements and specifications of the system are examined.
- Tester chooses valid inputs (positive test scenario) to check whether SUT processes them correctly. Also some invalid inputs (negative test scenario) are chosen to verify that the SUT is able to detect them.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs.
- The test cases are executed.
- Software tester compares the actual outputs with the expected outputs.
- Defects if any are fixed and re-tested.

Types of Black Box Testing

There are many types of Black Box Testing but following are the prominent ones -

- Functional testing – This black box testing type is related to functional requirements of a system; it is done by software testers.
- Non-functional testing – This type of black box testing is not related to testing of a specific functionality, but non-functional requirements such as performance, scalability, usability.
- Regression testing – Regression testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

6.2.2 WHITE BOX TESTING

White Box Testing is the testing of a software solution's internal coding and infrastructure. It focuses primarily on strengthening security, the flow of inputs and outputs through the application, and improving design and usability. White box testing is also known as clear, open, structural, and glass box testing.

It is one of two parts of the "box testing" approach of software testing. Its counter-part, blackbox testing, involves testing from an external or end-user type perspective. On the other hand, Whitebox testing is based on the inner workings of an application and revolves around internal testing. The term "whitebox" was used because of the see-through box concept. The clear box or whitebox name symbolizes the ability to see through the software's outer shell (or "box") into its inner workings. Likewise, the "black box" in "black box testing" symbolizes not being able to see the inner workings of the software so that only the end-user experience can be tested

What do you verify in White Box Testing ?

White box testing involves the testing of the software code for the following:

- Internal security holes
- Broken or poorly structured paths in the coding processes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object and function on an individual basis

The testing can be done at system, integration and unit levels of software development. One of the basic goals of whitebox testing is to verify a working flow for an application. It involves testing a series of predefined inputs against expected or desired outputs so that when a specific input does not result in the expected output, you have encountered a bug.

How do you perform White Box Testing?

To give you a simplified explanation of white box testing, we have divided it into **two basic steps**. This is what testers do when testing an application using the white box testing technique:

STEP 1) UNDERSTAND THE SOURCE CODE

The first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing. Also, the testing person must be highly aware of secure coding practices. Security is often one of the primary objectives of testing software. The tester should be able to find security issues and prevent attacks from hackers and naive users who might inject malicious code into the application either knowingly or unknowingly.

Step 2) CREATE TEST CASES AND EXECUTE

The second basic step to white box testing involves testing the application's source code for proper flow and structure. One way is by writing more code to test the application's source code. The tester will develop little tests for each process or series of processes in the application. This method requires that the tester must have intimate knowledge of the code and is often done by the developer. Other methods include manual testing, trial and error testing and the use of testing tools as we will explain further on in this article.

6.3 TESTCASES :

Below table shows the test case for the admin. The details if admin are stored in the database such as admin name, admin name etc. Admin is responsible for handling the transactions. The admin sets password and user name. Whether all the passwords and usernames are correct or not is checked. Because by using these details the user will login in into the cloud and can view the details of file stored.

Test case 1

The test case 1 tests the login of admin's. The passwords and username is given if the correct password and usernames are entered the login will be successful. If any wrong passwords and username the login will be denied. The test case 1 tests the login of admin's. The passwords and username is given if the correct password and usernames are entered the login will be successful. If any wrong passwords and username the login will be denied. Admin is responsible for handling the transactions. The admin sets password and user name. Whether all the passwords and usernames are correct or not is checked. Because by using these details the user will login in into the cloud and can view the details of file stored.

Sl # Test Case : -	UTC-1
Name of Test: -	Login as Admin
Items being tested:	Admin model
Sample Input: -	Correct Username & Password is given as inputs
Expected output: -	Depending on the correct inputs, it must login as Admin
Actual output: -	Login successful
Remarks: -	Pass.

Test case 2

The test case 2 is for checking the users. Again for the user's password and username is given based on only the correct input the users will be able to login

Sl # Test Case : -	UTC-2
Name of Test: -	Login as User
Items being tested: -	User model
Sample Input: -	Incorrect Username & Password is given as inputs
Expected output: -	Depending on the incorrect inputs, it shouldn't login as User
Actual output: -	Login Successful.
Remarks: -	Failed.

Test Case 3

Sl # Test Case : -	ITC-1
Name of Test: -	Send data to users
Item being tested: -	Data sent from attacker or user to be tested
Sample Input: -	Upload data
Expected output: -	Data sent through manager show attacker or normal packet
Actual output: -	Manager shows attacker or normal
Remarks: -	Pass.

Test case 3

The test case 3 shows for the file upload and it is being stored in different cloud which is called block generation and storing and hence this file is divided and stored .If the storage and block generation is not successful than we can store the file

Test Case 4

Sl # Test Case : -	STC-1
Name of Test: -	Complete task
Item being tested: -	Check data manager report
Sample Input: -	Click on send button from owner
Expected output: -	Data received at receiver side
Actual output: -	Data received
Remarks: -	Pass

Test case 4

Test case 4 is for file upload where the uploaded is downloaded. During download if any one of the cloud fails the recovery should start automatic. If the recovery is successful all the blocks will be generated and merged together. With the help of network coding the blocks are stored in the cloud.

CHAPTER: 7

RESULTS

7.1 SCREEN SHOTS:

View User Requests and Verify Attributes

Search keyword	Department	Profession	User	Hashcode	Status	Verify
java	General Services	Physician	ram@gmail.com	3254818	File Verified	Verify
doc	General Services	Physician	sreyas@gmail.com	99640	File Verified	Verify
word	General Services	Physician	navyadulam11@gmail.com	3655434	File Verified	Verify

Fig 7.1.1 User Requests

User Secret Key Verification

Secret key

Fig 7.1.2 Secret Key Verification

User keywordsearch Matched Files

File Name	Email	Keyword	Department	Profession	Hash_Code for Search Keyword	Cipher Data	Download
word	navyadulam11@gmail.com	word	General Services	Physician	3655434	XLFrwZfwR+AL+7IA3 hp+n4b68yahT7MIIdJj	Download

Fig 7.1.3 Matching of Keywords



Fig 7.1.4 Encrypted Data

CHAPTER: 8

CONCLUSION

In this paper, a fine-grained authorized expressive keyword search scheme has been proposed. The proposed scheme uses the benefits of CP-ABE to perform efficient keyword search over encrypted data and for authorizing users. In addition, it uses an interactive protocol for users to send trapdoors to the CSP, thus avoiding construction of secure channels. Further, it uses a bilinear-map accumulator for providing efficient user revocation, thereby avoiding the cumbersome re-encryption operations over the encrypted data. Conjunctive keyword search functionality is added by the proposed scheme at no extra overhead. The security analysis is performed under standard complexity assumptions and it shows that the proposed scheme is semantically secure against the chosen keyword attack. The scheme is also resistant against keyword guessing attacks. Moreover, the performance analysis of the proposed scheme shows that it outperforms the closely-related works in terms of storage, communication overhead, and computational overhead. Security, efficiency, and query expressiveness are the three main factors that define the practical usability of a keyword search scheme [4]. A user expects that the keyword search scheme provides strong security and high efficiency (in terms of computation and communication costs), as high as for normal plaintext search. Further, the query expressiveness will provide the user to make different types of queries. In this paper, we have made an effort to maintain a balance among these three factors. We believe that there is still much work to do for improving the balance among these three factors, i.e., security, efficiency, and query expressiveness. This will lead to the use of the keyword search schemes in privacy-preserving personalized services over the Internet like online advertisements or news..

CHAPTER 9

FUTURESCOPE

9.1 FUTURESCOPE

The concept of using authorized keywords over encrypted data in a cloud environment has significant potential for enhancing data security and privacy. Here's a look at some potential future directions and advancements in this area:

- **Homomorphic Encryption:** Advancements in homomorphic encryption techniques could enable more efficient processing of encrypted data in the cloud while maintaining privacy. This would allow authorized keywords to be searched over encrypted data without compromising confidentiality.
- **Secure Multi-Party Computation (SMPC):** SMPC protocols enable multiple parties to jointly compute a function over their inputs while keeping those inputs private. Integrating SMPC with keyword search functionalities could provide a robust method for searching over encrypted data in the cloud.
- **Blockchain Integration:** Integrating blockchain technology with encrypted data storage could enhance data integrity and auditability in cloud environments. Authorized keyword search transactions could be recorded on a blockchain, providing a transparent and tamper-proof log of data access.
- **Differential Privacy:** Techniques from the field of differential privacy could be applied to authorized keyword search to ensure that individual user's data remains private even when queried. This could involve adding noise to query results to prevent inference attacks.
- **Machine Learning and AI:** Advancements in machine learning and AI algorithms could enable more accurate and efficient searching over encrypted data. Techniques such as natural language processing (NLP) could be leveraged to improve the relevance of search results without revealing the underlying data.

- **Quantum Computing:** With the potential rise of quantum computing, there will be both opportunities and challenges for encrypted data processing. Quantum-safe encryption schemes will be essential to maintain security in the face of quantum attacks, while quantum computing itself could potentially be leveraged to improve the efficiency of encrypted data operations.

- **Standardization and Interoperability:** As encrypted data search mechanisms evolve, there will be a need for standardization and interoperability to ensure compatibility between different cloud platforms and services. Developing industry standards for authorized keyword search over encrypted data could help foster innovation and adoption.

- **Regulatory Compliance:** As data privacy regulations continue to evolve, there will be a growing need for solutions that enable compliance with laws such as GDPR and CCPA. Encrypted data search mechanisms that provide strong security guarantees while still allowing for authorized access could play a crucial role in meeting these compliance requirements.

Overall, the future scope for authorized keyword search over encrypted data in cloud environments is promising, with opportunities for innovation across various technological domains. However, addressing challenges such as efficiency, security, and regulatory

9.2 ADVANTAGES

Using authorized keyword search over encrypted data in cloud environments offers several advantages:

- **Privacy Protection:** Encrypted data ensures that sensitive information remains private, even when stored or processed in the cloud. Authorized keyword search allows users to retrieve specific information without revealing the content of the encrypted data to the cloud service provider or any unauthorized parties.

- **Data Confidentiality:** Encrypted data remains confidential throughout its lifecycle in the cloud. Authorized keyword search enables users to query encrypted data without decrypting it, thereby preserving confidentiality and reducing the risk of unauthorized access or data breaches.
- **Secure Data Outsourcing:** Cloud computing allows organizations to outsource their data storage and processing needs. Authorized keyword search over encrypted data enables organizations to leverage cloud services without compromising the security and privacy of their sensitive information.
- **Selective Access Control:** Authorized keyword search enables fine-grained access control over encrypted data. Users can specify which keywords they are authorized to search for, ensuring that only relevant data is accessed while preventing unauthorized access to other parts of the data.
- **Reduced Trust Requirements:** Authorized keyword search reduces the trust requirements placed on cloud service providers. Since data remains encrypted throughout the search process, users do not need to fully trust the cloud provider with access to their plaintext data, enhancing security and mitigating the risk of insider threats.
- **Compliance with Regulations:** Many industries are subject to strict data privacy regulations that mandate the protection of sensitive information. Authorized keyword search over encrypted data helps organizations comply with regulations such as GDPR, HIPAA, and PCI-DSS by ensuring that sensitive data is stored and processed securely in the cloud.
- **Efficient Data Retrieval:** Despite the encryption, authorized keyword search allows for efficient retrieval of relevant information from large datasets. Users can search for specific keywords without the need to decrypt and process the entire dataset, resulting in faster search times and improved performance.
- **Flexibility and Scalability:** Authorized keyword search can be implemented in a scalable manner, allowing organizations to search over large volumes of encrypted data efficiently. This scalability ensures that the solution can grow with the organization's data needs without sacrificing performance or security.

- **Cross-Platform Compatibility:** Authorized keyword search solutions can be designed to work across different cloud platforms and services, providing flexibility and interoperability for organizations that use multiple cloud providers or migrate between different cloud environments.

Overall, authorized keyword search over encrypted data in cloud environments offers a powerful combination of privacy, security, and efficiency, making it an attractive solution for organizations looking to leverage cloud computing while protecting sensitive information.

9.3 DISADVANTAGES

While authorized keyword search over encrypted data in cloud computing offers numerous advantages, there are also some potential disadvantages and challenges to consider:

- **Complexity of Implementation:** Implementing authorized keyword search over encrypted data requires sophisticated cryptographic techniques and algorithms. Integrating these techniques into existing cloud systems can be complex and may require specialized expertise.
- **Performance Overhead:** Cryptographic operations involved in encrypted keyword search can introduce performance overhead. Searching over encrypted data may be slower compared to searching over plaintext data, especially for large datasets or complex queries.
- **Key Management:** Managing encryption keys securely is crucial for the effectiveness of authorized keyword search. Key management becomes more complex in a cloud environment, where keys may need to be shared among multiple users or services while ensuring they remain secure from unauthorized access.
- **Limited Search Capabilities:** Searching over encrypted data imposes limitations on the types of queries that can be performed. Complex search operations, such as pattern matching or fuzzy searching, may be challenging or inefficient to implement with encrypted data.

- **Risk of Insider Attacks:** While authorized keyword search reduces the risk of unauthorized access by external parties, it does not eliminate the risk of insider attacks. Malicious insiders with access to encryption keys or sensitive system components could potentially compromise the security of the encrypted data.
- **Scalability Challenges:** As the volume of encrypted data grows, scalability becomes a challenge. Efficiently searching over large datasets while maintaining acceptable performance levels requires careful optimization and resource allocation.
- **Potential for Metadata Leakage:** Even though the data itself is encrypted, metadata associated with search queries or access patterns could leak sensitive information. Protecting metadata privacy requires additional safeguards and considerations.
- **Interoperability Issues:** Different encryption schemes and protocols may not be interoperable across various cloud platforms and services. This lack of interoperability can hinder data portability and vendor lock-in.
- **Regulatory Compliance Complexity:** While encrypted data can enhance privacy and security, it can also complicate regulatory compliance efforts. Ensuring compliance with data protection regulations may require additional measures and documentation to demonstrate the effectiveness of encryption and access control mechanisms.
- **Trade-off Between Security and Functionality:** Implementing encrypted keyword search involves a trade-off between security and functionality. Striking the right balance between privacy, security, and usability requires careful consideration of the organization's specific requirements and risk tolerance.

Addressing these challenges requires a comprehensive approach that considers both technical and operational aspects of encrypted keyword search in cloud environments. Organizations must carefully evaluate the benefits and drawbacks to determine if authorized keyword search is the right solution for their needs.

CHAPTER: 10

REFERENCES

- N. Kaaniche and M. Laurent. Data security and privacy preservation in cloud storage environments based on cryptographic mechanisms. *Computer Communications*, 111:120–141, 2017.
- J. Tang, Y. Cui, Q. Li, K. Ren, J. Liu, and R. Buyya. Ensuring security and privacy preservation for cloud data services. *ACM Computing Survey*, 49(1):13:1–13:39, Jun 2016.
- F. Han, J. Qin, and J. Hu. Secure searches in the cloud. *Future Generation Computer Systems.*, 62(C):66–75, Sep. 2016.
- C. Bösch, P. Hartel, W. Jonker, and A. Peter. A survey of provably secure searchable encryption. *ACM Computing Survey*, 47(2):18:1–18:51, Aug 2014.
- D. X. Song, D. Wagner, and A. Perrig. Practical techniques for searches on encrypted data. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, SP '00, pages 44–, 2000.
- R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, CCS '06, pages 79–88, 2006.
- W. Sun, B. Wang, N. Cao, M. Li, W. Lou, Y. T. Hou, and H. Li. Privacy-preserving multi-keyword text search in the cloud supporting similarity-based ranking. In *Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security*, ASIA CCS '13, pages 71–82, 2013.
- N. Cao, C. Wang, M. Li, K. Ren, and W. Lou. Privacy-preserving multi-keyword ranked search over encrypted cloud data. *IEEE Transactions on Parallel and Distributed Systems*, 25(1):222–233, Jan 2014.
- C. Guo, X. Chen, Y. Jie, F. Zhangjie, M. Li, and B. Feng. Dynamic multi-phrase ranked search over encrypted data with symmetric searchable encryption. *IEEE Transactions on Services Computing*, pages 1–1, 2017.
- N. H. Sultan and F. A. Barbhuiya. A secure re-encryption scheme for data sharing in unreliable cloud environment. In *Proceedings of the 2016 IEEE World Congress on Services (SERVICES)*, pages 75–80, June 2016.
- W. Sun, S. Yu, W. Lou, Y. T. Hou, and H. Li. Protecting your right: Attribute-based keyword search with fine-grained owner-enforced search authorization in the cloud. In *Proceedings of the 2014 IEEE Conference on Computer Communications*, IEEE INFOCOM, pages 226–234, April 2014.
- 4shared. <https://www.4shared.com/> [Online accessed: 16-Dec. 2018].
- MediaFire. <https://www.mediafire.com/> [Online accessed: 16 Dec.-2018]. [14] MEGA. <https://mega.nz/> [Online accessed: 16-Dec.-2018].
- K. Yang and X. Jia. Expressive, efficient, and revocable data access control for multi-authority cloud storage. *IEEE Transactions on Parallel and Distributed Systems*, 25(7):1735–1744, July 2014.

- S. Ruj, M. Stojmenovic, and A. Nayak. Decentralized access control with anonymous authentication of data stored in clouds. *IEEE Transactions on Parallel and Distributed Systems*, 25(2):384–394, Feb 2014